CITS5501 Phase 2

Group details

Name	Student Id	GitHub Username
Faiz Qazi	25040556	faizqazi
Zhenhao (Howard) Jiang	24733021	skybord32222
Kaili Zhou	24057973	KylieZhou
Md Irtiza Anam	25086045	Irtizaanam
Tony Chu	24074951	TonyChyu

GitHub repository link: https://github.com/skybord32222/CITS5501-26S2-Team22

1. Task 1 - Prototype

1.1 Purpose and implementation

Our prototype is a simple Python script that checks if policy documents can be found and downloaded from the UWA policy index page provided in the brief. It is just a basic proof of concept and is not designed to crawl the whole website.

The script starts by downloading the index page and collecting all the links using Python's built-in `HTMLParser`. Instead of searching for specific CSS classes, we check all the links so the script can still work if the page layout changes. Each link is converted into a full URL, and anything after the # symbol is removed. Links to other websites and links that are not normal web links are skipped. We then keep only links to supported document formats, including PDF, DOC, DOCX, XLS, XLSX, PPT, PPTX, ODT, ODS, ODP and RTF. Any duplicate links are removed.

By default, the script downloads the first 3 documents in the list. The number of files can be changed with `--count`, allowing us to download 3, 4 or 5 documents. For each document, the script shows its URL, HTTP status code, content type, size in bytes and the first few bytes of the file. We do not save the downloaded files to the computer. The script holds the data in memory while running the checks and discards it once they are complete.

We used only Python's standard library, including `html.parser`, `urllib` and `json`, so there is no need to install extra packages. The script does not crawl through other pages or check the actual contents of the downloaded files, as these features are not required by the brief.

We use type annotations to document function inputs and outputs wherever possible, including a TypedDict to describe download metadata. These make interfaces clearer and support static checking, but do not enforce types at runtime or validate downloaded documents.

1.2 Request and resource limits

For our run we used `--count 3`, so the script only made 4 requests: one for the index page and one for each document. Even if you turn it up to `--count 5`, that's still at most 6 requests, which is well under the brief's example budget of 20. We didn't add retries and the script doesn't follow redirects, so you always know exactly how many requests it will make. The catch is that if a document has been moved behind a redirect, the script just counts it as a failure.

We gave each request a 5-second timeout so the script won't get stuck forever if the server stops responding. That said, it's not a limit on the whole download, so a slow download that keeps trickling in data could still go past 5 seconds. We also capped downloads at 5 MB, and anything bigger is marked as failed. If one document doesn't work, the script just moves on to the next; the only thing that stops it completely is not being able to load the index page.

It's worth pointing out that a "successful" download here only means the full file came through within those limits. It doesn't tell us whether the file actually opens properly as a PDF or office document.

1.3 Run evidence

The results below come from the recorded run that was made while the prototype was being developed (`python3 scraper.py --count 3`, Python 3.14.7, 13 October 2026 at 10:29 UTC). This section summarises that run rather than a new run by the writer. The script found 216 different candidate links on the index page, tried 3 of them, and all 3 downloads finished.

Document / exact policy URL	HTTP status	Declared type	Bytes read	Initial-byte prefix	Interpretation	Evidence
policy-governance-framework.pdf	200	application/pdf	301,201	`%PDF-1.6`	Body download completed; PDF-like metadata; usability not validated	Record 1
up26-2-code-of-conduct.pdf	200	application/pdf	163,909	`%PDF-1.6`	Body download completed; PDF-like metadata; usability not validated	Record 2
up26-2-code-of-conduct—annexure-a.pdf	200	application/pdf	217,240	`%PDF-1.6`	Body download completed; PDF-like metadata; usability not validated	Record 3

All three files returned status 200, were reported as `application/pdf`, and started with the `%PDF` signature, so they appear to be genuine PDFs, consistent with the PDF format. However, we did not open them in a PDF viewer, so we cannot say for certain that they are complete, usable files.

1.4 Assumptions and limitations

To keep the code simple, we assumed that all policy documents are linked on the index page and only looked at links from the same website. We also used file extensions to identify documents, as this was allowed in the brief.

Limitation	Why we did it this way	What could go wrong	How it could be improved
Documents are found by file extension only	Provides a lightweight way to select likely policy documents	A policy at a link like `/download?id=17` would be missed, and a `.pdf` link that actually returns a web page would still be picked	Consider extensionless links by using endpoint patterns, and inspect response headers/contents with a reasonable bound
Files are not checked to see if they open	Keeps the prototype minimal	A login page or a broken PDF sent with status 200 would still count as "downloaded"	Open each file with a PDF or office file reader before accepting it
Redirects are not followed	Keeps the number of requests fixed	A real document that has moved behind a redirect would be reported as failed	Allow a small number of redirects as long as they stay on the same site

2. Task 2 - Design of test suite

2.1 Reasonable production expectations

We designed these tests to check how the policy document scraper should handle different errors in a production system. Compared with our current prototype, we expect the final system to handle errors better, retry failed requests, and check whether downloaded documents are valid.

For our tests, we suggest setting a one-second timeout for each request and allowing up to three attempts if a temporary error occurs. We also suggest waiting 0.1 and 0.2 seconds between retries. To avoid the program running for too long, we set a ten-second overall time limit. These are the rules we propose for testing, and they are not necessarily implemented in our current prototype.

If we cannot access the policy index after three attempts, the scraper should stop and report the error, because it cannot find the document links. However, if only one document fails, we expect the scraper to record the error and continue with the remaining documents.

For incomplete HTML, we only want the scraper to use complete and valid links. If the HTML is clearly incomplete, the scraper should also report this problem. We also need to check the downloaded files, because a successful HTTP response does not always mean the file is valid. For PDF and Office files, we suggest using suitable validation tools to check the file format before accepting them.

2.2 Test-case table

The following eight test cases cover all five scenarios specified in the project brief. Each case defines the test setup, expected behaviour, execution steps, and observable results. These tests describe proposed production behaviour rather than features already implemented in the prototype.

ID	Supplied scenario	Input/fixture seed	Expected behaviour	Arrange details	Act	Assert / observable evidence
T001	1 - Structure changes	Same policy anchors moved into new containers/ classes	The scraper should identify the same policy links despite changes to the HTML layout.	Prepare two HTML pages containing the same three PDF links but using different containers and CSS classes.	Run document discovery on both pages.	Both pages produce the same expected set of three URLs, with no missing or additional links.
T002	2 - Truncated HTML	One complete anchor followed by an unfinished anchor attribute	The scraper should report detectable incomplete input and avoid generating invalid URLs.	Prepare an HTML response containing one complete PDF link and a second unfinished href attribute. Simulate an interrupted response to make truncation detectable.	Run index retrieval and parsing.	Incomplete input is reported, exactly the first PDF URL is discovered, and no URL is generated from the unfinished attribute.
T003	3 - Temporary timeout	Document request times out once, then returns a valid PDF	The scraper should retry a temporary failure and successfully download the document.	Configure a mock request to time out once and return a valid PDF on the second attempt. Use a fake clock to record retry delays.	Run the document download operation with retries enabled.	Exactly two attempts are made, one 0.1-second retry delay is recorded, and the valid PDF is accepted.
T004	3 - Persistent timeout	Every index request times out	The scraper should stop after the maximum permitted attempts and report the failure.	Configure the mock index request to time out on every attempt, with a maximum of three attempts.	Run index retrieval.	Exactly three attempts are made, a timeout failure is reported, and no document download requests occur.
T005	4 - 4xx status	One document always returns 404; valid control succeeds	The scraper should reject unavailable document	Configure one document URL to return HTTP 404 and another	Run document downloads.	The 404 document is rejected after one attempt, its

ID	Supplied scenario	Input/fixture seed	Expected behaviour	Arrange details	Act	Assert / observable evidence
			without retrying and continue processing other documents.	to return a valid PDF.		failure is recorded, and the valid PDF is accepted.
T006	4 - 5xx status	One document always returns 503, without Retry-After	Retry within the configured limit, then report failure and continue.	Configure one URL to always return 503 and another to return a valid PDF. Use a fake clock to record delays.	Run document downloads with retries enabled.	Exactly three attempts for the failing URL and delays of 0.1 and 0.2 seconds are recorded. The failed document is rejected and the valid PDF is accepted in a separate request.
T007	5 - HTML for PDF URL	HTTP 200 + text/html + login HTML at a .pdf URL	Reject the HTML response and continue processing other documents.	Configure a .pdf URL to return HTML with status 200 and another URL to return a valid PDF.	Run document retrieval and validation.	The HTML response is rejected without retrying, a format-mismatch reason is recorded, and the valid PDF is accepted.
T008	5 - Corrupt PDF	HTTP 200 + application/pdf + %PDF prefix but unusable contents	Reject the corrupt PDF despite its plausible header.	Prepare independently checked corrupt and valid PDF fixtures, both returned with status 200 and application/pdf.	Run document retrieval and PDF validation.	The corrupt PDF is rejected with a validation reason, the valid PDF is accepted, and processing completes without crashing.

2.3 Coverage and rationale

We designed eight test cases to cover all five required scenarios. In T001 and T002, we test how the scraper handles changes or incomplete content in the policy index. T003 and T004 focus on temporary network errors and what happens when requests keep timing out. For T005 and T006, we check how the scraper handles different HTTP errors, such as 404 errors that should not be retried and 503 errors that can be retried. Finally, T007 and T008 test whether the scraper can identify invalid documents, such as HTML responses instead of actual files or corrupted PDF files.

We suggest setting a retry limit to avoid sending too many requests, while still giving the scraper a chance to recover from temporary errors. We also expect the scraper to continue processing other documents if one document fails.

To make our tests consistent and repeatable, we plan to use fixed HTML samples, valid and invalid documents, mocked HTTP responses, request logs, and a fake clock. This allows us to check the expected results without depending on the live website.

These eight tests cover the five required scenarios, but they do not cover every possible problem. In the future, we could add more tests for redirects, duplicate documents, unusual file formats, and cases where the system runs out of resources.

All eight test cases are proposed for a production version of the scraper. We have not implemented or successfully tested them against our current Python prototype.

3. Task 3 - Fixtures and test doubles

3.1 Oracles and Repeatability

Fixtures are the data we can control, test doubles are stand-ins for the collaborator we cannot control, and an "oracle" is what we compare against-it must be independent of the code under test. The live site cannot serve as our oracle: its links and availability change at any time; more importantly, we cannot make a third-party server "deliberately" return a bad file, hang without responding, or reject us. Therefore, all oracles below are pinned to frozen local bytes and scripted responses.

Fixture / Test Double	Supported Scenarios	Design and Expected Results	Reset / Repeatability
Fixed index page `task3/1/www/index.html`	Scenario 1 (index structure changes, link discovery, and extension filtering)	Handwritten index: 6 policy links (`good6.pdf` deliberately duplicated twice), plus `fake.pdf`, `slow.pdf`, `hang.pdf`, and an off-site `.jpg`. Expected: after deduplication and extension filtering, **exactly 9** internal document URLs (`good1`–`good6`, `fake`, `slow`, `hang`), with duplicates merged and the `.jpg` removed	Static file, fixed bytes, no reset needed
Truncated index page `task3/1/www/truncated.html`	Scenario 2 (HTML download is truncated midway)	A truncated prefix of the real index body (`task3/2/body1`), with the cut point immediately after an `<a>` opening tag, the final element unclosed, and no `</body></html>`. Expected: **exactly 31** valid URLs parsed, all of which are a subset of the complete page's 216 URLs; no crashes and no fabricated URL for the dangling element	Static file, fixed bytes
Valid / invalid PDF pair: `good1.pdf` (301,201 B), `fake.pdf` (43 B), `corrupt.pdf` (293 B)	Scenario 5 (link does not return a well-formed PDF)	`good1` is a genuine PDF (`%PDF-1.6` + xref + `startxref` + `%%EOF`); `fake` is named `.pdf` but is actually HTML; `corrupt` retains the PDF file header but has an unusable xref and no `startxref` or `%%EOF`. Independent fixture check: Poppler's `pdfinfo` accepts `good1` and rejects `fake` and `corrupt`. Magic numbers and trailer markers are preliminary checks, not proof of general PDF usability	Static files, fixed bytes
Scripted local HTTP service `task3/1/server.py` (standard library only)	Scenario 4 (HTTP error responses), Scenario 3 (timeout / very slow response)	Serves the fixture directory by path: the 7th and subsequent in-process `.pdf` requests return **429**, missing paths return **404**, `/slow.pdf` waits a fixed **15 s** before replying, and `/hang.pdf` holds the connection for **3600 s** without responding. With `curl --max-time 5`, both delayed endpoints produce exit code **28**. These fixed delays exceed the configured client timeout.	Restart the process to clear the request count; on-disk fixtures remain unchanged

Fixture / Test Double	Supported Scenarios	Design and Expected Results	Reset / Repeatability
		The separate windowed fixture is described in section 3.2.	
Offline stubbed test double `FakeResponse` + patched `build_opener` in `test_scraper.py`(from task 1-2)	The interrupted-download path in Scenario 3 and the client-side error path (Task 1 output, referenced here only)	The injected TimeoutError, URLError and IncompleteRead exceptions produce a failed outcome and a recorded error. The HTML FakeResponse instead produces downloaded (not format-validated), demonstrating the missing validation step. Each tested fetch makes one mocked opener call, without retries or real network traffic.	The setup restores the real opener after the test case; no sockets are created throughout

3.2 Live Observations and Request Limiting

To investigate request limiting, we used a shell script to make a bounded sequence of requests to the policy index and save each response's headers and body for inspection.

We reviewed the evidence produced by the script. The script issued 10 consecutive `curl` requests to the policy index without waiting between them, so in total we issued 10 requests, all targeting the same index URL, serially and without waiting. Requests 1–6 returned **200**, with `Content-Length: 584187`; requests 7–10 returned **429**, with `Content-Length: 40740`.

Under **these** conditions-10 serial requests, no waiting, and only one index URL targeted-the site first returned six successful responses and then rejected us with **429 Too Many Requests**. The rejection was explicit and machine-readable. Every response included `server: cloudflare`, `x-ratelimit-limit: 3`, an `x-ratelimit-remaining` value that steadily decreased to zero, and a `retry-after` that decreased from 10 seconds to 3 seconds as the window advanced. Two points are important for testing. First, the **declared threshold did not match the actual observation** (the header said 3, yet we obtained six consecutive successes), so the advertised limit alone does not establish the enforcement policy. These observations do not distinguish between possible rate-limiting algorithms; the supported conclusion applies to **the observed sequence**, not to the configured threshold. Second, the 429 body was neither empty nor a JSON error; it was an **HTML page of 40,740 bytes** (titled "429 Too Many Requests") that still contained 152 `href` attributes, but **zero links with document extensions**. We measured the consequence: feeding this 429 body into the link-discovery step yielded **0** candidate documents, whereas the real index contained 216. In other words, **a crawler that looks only at the body and ignores the status code will "succeed" on a rate-limited response and report that no policy documents were found**.

Several things remain unknown. The exact threshold and window length-we stopped after 10 requests (following the task brief's ten-attempt guidance), so we have no data beyond 10 and cannot characterize behavior under higher concurrency or bursts; whether rate limiting is applied per IP, per path, or globally; whether document URLs are subject to the same limit as the index (we probed only the index); whether the limiting escalates into a longer ban; and how the service recovers after `retry-after` is honored.

The live site advertised a threshold of 3 that did not match the number of successes we actually obtained (6), so that advertised limit alone cannot establish the live server's enforcement policy. `task3/1/server_windowed_429.py` therefore reproduces the mechanics we observed - rate-limit headers on every response, a Retry-After countdown, a non-empty 429 body, and a quota that is restored once the window passes - while fixing the threshold to a deliberately small, easily triggered value of 3 requests per 10 s window. It does not attempt to reproduce the site's exact 3-versus-6 behaviour, which we could not characterise within the 10-request budget; it fixes the shape of the response and leaves the threshold to whichever test selects it.

The reason why we did not perform more intensive probing was because the task brief states that it may be possible to identify rate limiting in no more than 10 download attempts, and we used exactly 10. Going further would amount to stress-testing a shared third-party server. Yet we had already obtained the answers we needed (whether rate limiting exists, whether it is observable, and whether it is machine-readable). The threshold itself is a configuration property of the site's operator, not something required to write repeatable tests.

Our test design avoids depending on the live site. Proposed local integration checks use `http://127.0.0.1:8000` and fixed expectations: 9 URLs in the handwritten index, 31 URLs in the truncated index, independently checked PDF controls, and scripted 429 / 404 / slow / hang responses. The submitted offline unit tests instead patch HTTP access and make no real network requests. Restarting the local services clears their request state; the windowed fixture also restores quota when its window expires. Resetting state and reusing frozen fixtures allows repeated checks of **our own behavior**, rather than the site's changing content. The bounded live investigation described above is separate from the offline suite and is not required to run the submitted tests. Any additional manual smoke check should use few requests and respect `Retry-After`.

3.3 Failure simulation tools

Python's `unittest.mock` allows network failures to be simulated without relying on the live policy website. We replace the HTTP opener used by the scraper and configure its responses or exceptions. This makes individual failure conditions repeatable and allows us to inspect outcomes and request counts.

Failure	Simulation and assertion	Current coverage
Timeout	Use mock side_effect to raise TimeoutError. Assert a failed outcome and recorded reason; a production retry test would also assert bounded attempts and delays.	Exception-handling path implemented; production retries only proposed.
Connection failure	Raise URLError from the mocked opener. Assert failure is reported without an uncaught exception.	Implemented as a mocked failure, not a real socket test.
Interrupted download	Raise IncompleteRead while reading the response body. Assert the download is not reported as completed.	Implemented as a mocked interrupted read.
Very slow response	Use a controlled local server that sends data at intervals shorter than the socket timeout. Assert termination within a separately specified overall deadline, with an independent watchdog.	Proposed integration test; not currently implemented.

These doubles check how the scraper responds to controlled failures, but they do not establish real network timing or cover every interrupted-transfer condition. A local server would provide stronger integration coverage because it exercises actual socket and streaming behaviour. Each test should reset its mock responses and request history; local-server tests should also close connections and stop the server during cleanup.

For proposed retry tests, an injected clock and sleeper would allow delays to be checked without waiting in real time. The current prototype does not implement retries or an overall download deadline, so those assertions describe production expectations rather than existing functionality.

3.4 Provided failure artifact and interpretation

Our failure artifact is `test_double_exposes_missing_format_validation`, together with `FakeResponse`, in test_scraper.py. It simulates a policy URL ending in `.pdf` that returns HTTP 200, a declared content type of `text/html`, and an HTML login page instead of a PDF.

The test patches `urllib.request.build_opener`, which is the factory called by `fetch`. The replacement opener returns the controlled response without contacting the network. The assertions verify the status and content type, confirm that the body does not begin with `%PDF`, and check that the prototype reports `downloaded (not format-validated)`.

This is a **passing diagnostic test that demonstrates a validation gap**, rather than evidence that the scraper correctly rejects invalid documents. Its oracle deliberately checks the prototype's current behaviour: it reports completion of the response body without establishing document usability. A production acceptance test should instead require the HTML response to be rejected as a policy document, with a clear validation reason.

The specific demonstration can be reproduced using:

```
python3 -m unittest discover -s tests -p test_scraper.py -k
test_double_exposes_missing_format_validation -v
```

The full offline suite passed thirteen checks during development, as documented in the recorded test output. These are prototype regression checks, not an implementation of the entire proposed production suite.

Reproducing this condition against the actual policy website would require control over its responses or dependence on an unpredictable error condition. The double avoids modifying or burdening the live service and supplies the same response on every run, making the demonstrated limitation reproducible.

4. Task 4 - Adversarial inputs

4.1 Reasoning and scope

Adversarial testing examines how a web scraper handles unexpected inputs and irregular situations. While a web scraper may work fine under ideal conditions, it will likely be written with a set of assumptions about document URLs, website structure, redirects, download behavior, and file handling. The following six test cases explore these assumptions using controlled HTML fixtures, sample documents, and local test servers. For each test, we identify the assumption that the test explores, a test scenario, and observable evidence of failure.

Idea ID	Assumption challenged	Input/environment seed	Exact artifact/setup to use	Failure criterion and detector	Scope qualification
A001	Policy documents always have recognised filename extensions.	An extensionless URL, /download?id=17, returns a valid pdf.	Create a local html index linking to this endpoint and a standard .pdf document. Configure them both to return a valid pdf content.	The scraper misses the extensionless document when a broader discovery is expected. Compare discovered urls against a predefined list of valid document links.	Extension based filtering is permitted for the Task 1 prototype; this tests a broader document discovery.
A002	Different URLs always represent different policy documents.	/policy.pdf and /policy.pdf?ref=archive return identical PDF content.	Configure local server to return identical PDF bytes for both URLs. Record requests and calculate SHA-256 hashes of the downloaded content.	If content-level deduplication is required, then this test fails, because either one document has been counted twice or is downloaded twice. This could be detected by comparing document hashes and download records.	Tests content-level deduplication; prototype only removes duplicate URLs.
A003	Links originating from the target website are always on the target website.	A document URL returns an HTTP 302 redirect to a different origin.	Set up two local servers. The first redirects to the second one, which then logs the requests.	There is an unexpected entry in the second server's logs, indicating that the scraper followed the redirect to the wrong origin.	Testing redirect security; the current prototype does not follow redirects.

Idea ID	Assumption challenged	Input/environment seed	Exact artifact/setup to use	Failure criterion and detector	Scope qualification
A004	Relative links can always be resolved against the index URL.	An HTML page contains `<base href="/documents/">` and a relative link to policy.pdf.	Configure / documents/policy.pdf as the valid file and / policy.pdf as an invalid endpoint on a local server.	Scraper requests wrong url or fails to find valid document, check requested path against / documents/policy.pdf	Test html base-element handling that the current prototype doesn't implement
A005	A socket timeout limits the total download duration.	A server continuously sends small chunks before each socket timeout expires.	Configure a local streaming server that exceeds a chosen overall deadline. Record elapsed time using a monotonic clock.	The scraper continues beyond the required overall deadline. Detect this using elapsed-time measurements and an independent test watchdog.	Test the total duration limits. The prototype scraper has a socket timeout but no overall deadline for the download.
A006	Server-provided filenames can be used safely for saving files.	A response filename contains ../../escape.pdf.	Run a saving scraper inside an isolated temporary directory, with a protected file outside the download folder. Check the final resolved paths and file hashes.	A file is written outside the allowed directory, or a protected file is overwritten. Can be detected by resolved paths, and comparing before/after hashes.	Applies to scrapers that save files. The current prototype holds downloaded data in memory.

These six adversarial tests cover assumptions about document discovery, duplicate content, redirects, relative URLs, download duration and storage of files. They are written using carefully selected input and specific failure conditions that can be reproduced deterministically without impacting the live policy website. However, some of these tests explore features beyond the minimal scope of Task 1, and should therefore be considered as reliability tests for a wider class of scrapers in the current implementation.

References

[1] CITS5501/CITS3501 Project 2026 Phase 2. (2026).